3-Tier Web Application on Kubernetes

Deployed on a Minikube cluster running on an AWS EC2 instance

Project Documentation

Stack	Details
Cloud	AWS EC2 (Ubuntu)
Orchestration	Kubernetes via Minikube
Frontend/App tier	Node.js app — `nanajanashia/k8s-demo-app:v1.0`
Database tier	MongoDB — `mongo:latest`
Proxy tier	Nginx reverse proxy on the EC2 host

1. Project Overview

This project deploys a three-tier web application on a single-node Kubernetes cluster. The cluster was created with Minikube on an AWS EC2 instance, and all application components were deployed from YAML manifests using `kubectl apply`. The application is a simple **User Profile** page whose data is stored in MongoDB.

The three tiers

Tier	Component	Kubernetes objects
Presentation / Proxy	Nginx on EC2 host (port 80)	Runs outside the cluster; forwards to the NodePort
Application	Node.js web app	Deployment `webapp-deployment`, Service `webapp-service` (NodePort 30100)
Database	MongoDB	Deployment `mongo-deployment`, Service `mongo-service` (ClusterIP 27017)
Configuration	Shared settings	ConfigMap `mongo-config`, Secret `mongo-secret`

Request flow

```
Browser --> EC2 public IP :80 --> Nginx (reverse proxy)
        --> Minikube IP :30100 (NodePort: webapp-service)
        --> webapp pod :3000
        --> mongo-service :27017 (ClusterIP) --> mongo pod
```

2. Environment Setup

2.1 EC2 instance

An Ubuntu EC2 instance was launched in the AWS console (region ap-south). Its security group must allow inbound **SSH (22)** for administration and **HTTP (80)** so the browser can reach Nginx.

2.2 Minikube cluster

Docker, kubectl and Minikube were installed on the instance, and a cluster was started. Typical commands for this step:

```
minikube start --driver=docker
kubectl get nodes
minikube ip          # e.g. 192.168.49.2 (used later in the Nginx config)
```

The Minikube node IP (`192.168.49.2`) is on a private Docker network, so the NodePort is reachable from the EC2 host but not directly from the internet. That is why Nginx is used in Section 5.

3. Kubernetes Manifests

All four manifests live in the project folder `3-Tier-Demo-Kube-App`. They are applied in dependency order: Secret, ConfigMap, MongoDB, then the web app.

3.1 mongo-secret.yaml

Stores the MongoDB root credentials. Values are base64-encoded (`YWRtaW4=` decodes to `admin`).

```
apiVersion: v1
kind: Secret
metadata:
  name: mongo-secret
type: Opaque
data:
  mongo-user: YWRtaW4=
  mongo-pass: YWRtaW4=
```

3.2 mongo-config.yaml

Holds the non-sensitive database host name. It equals the MongoDB Service name, so the web app resolves it through cluster DNS.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: mongo-config
data:
  mongo-url: mongo-service
```

3.3 mongo.yaml (Deployment + Service)

Runs one MongoDB pod, initialising the root user from the Secret. The Service is of type ClusterIP (the default), so MongoDB is only reachable inside the cluster.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongo-deployment
  labels:
    app: mongo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongo
  template:
    metadata:
      labels:
        app: mongo
    spec:
      containers:
      - name: mongo
        image: mongo:latest
        ports:
        - containerPort: 27017
        env:
          - name: MONGO_INITDB_ROOT_USERNAME
            valueFrom:
              secretKeyRef:
                name: mongo-secret
                key: mongo-user
          - name: MONGO_INITDB_ROOT_PASSWORD
            valueFrom:
              secretKeyRef:
                name: mongo-secret
                key: mongo-pass
---
apiVersion: v1
kind: Service
metadata:
  name: mongo-service
spec:
  selector:
    app: mongo
  ports:
    - protocol: TCP
      port: 27017
      targetPort: 27017
```

3.4 web-app.yaml (Deployment + Service)

Runs the Node.js app on port 3000. The database user and password come from the Secret, and the database host from the ConfigMap. The Service is of type **NodePort** and exposes the app on port 30100 of the node.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp-deployment
  labels:
    app: webapp
spec:
  replicas: 1
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
      containers:
      - name: webapp
        image: nanajanashia/k8s-demo-app:v1.0
        ports:
        - containerPort: 3000
        env:
          - name: USER_NAME
            valueFrom:
              secretKeyRef:
                name: mongo-secret
                key: mongo-user
          - name: USER_PWD
            valueFrom:
              secretKeyRef:
                name: mongo-secret
                key: mongo-pass
          - name: DB_URL
            valueFrom:
              configMapKeyRef:
                name: mongo-config
                key: mongo-url
---
apiVersion: v1
kind: Service
metadata:
  name: webapp-service
spec:
  type: NodePort # external service
  selector:
    app: webapp
  ports:
    - protocol: TCP
      port: 3000
      targetPort: 3000
      nodePort: 30100 #Range is 30000 -> 32767
```

Configuration wiring

Env var in web app	Source	Key
USER_NAME	Secret `mongo-secret`	mongo-user
USER_PWD	Secret `mongo-secret`	mongo-pass
DB_URL	ConfigMap `mongo-config`	mongo-url

Screenshot: web-app.yaml as edited on the server



Figure 1 — web-app.yaml opened in nano on the EC2 instance

4. Deployment

The manifests were applied in this order, so that the Secret and ConfigMap exist before the pods that reference them:

```
kubectl apply -f mongo-secret.yaml
kubectl apply -f mongo-config.yaml
kubectl apply -f mongo.yaml
kubectl apply -f web-app.yaml

kubectl get pods
kubectl get svc
```

Result

Both pods moved from *ContainerCreating* to *Running* (1/1 ready) within about 30 seconds. `kubectl get svc` shows `mongo-service` as ClusterIP on 27017 and `webapp-service` as NodePort mapping 3000:30100.

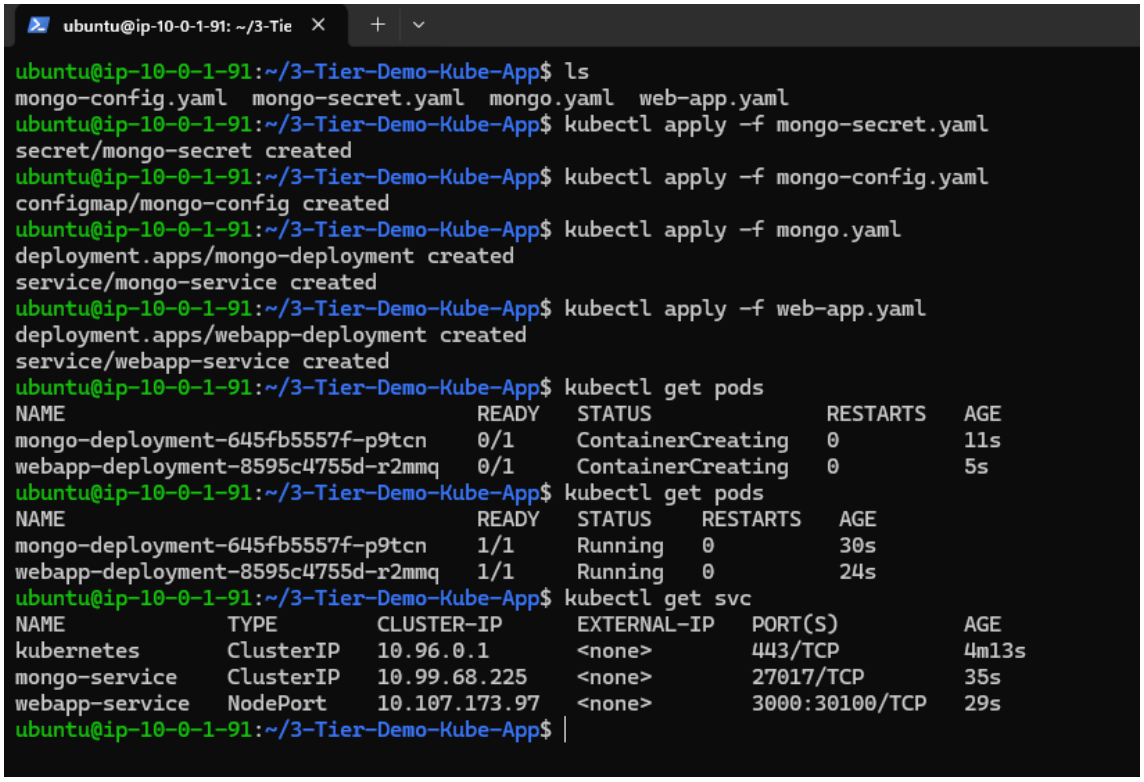


Figure 2 — Applying the manifests and verifying pods and services

5. Exposing the App with Nginx

Because the NodePort lives on the Minikube node's private IP, an Nginx reverse proxy was installed on the EC2 host. It listens on port 80 and forwards all traffic to `192.168.49.2:30100`. The config file is `/etc/nginx/sites-available/k8s-app`:

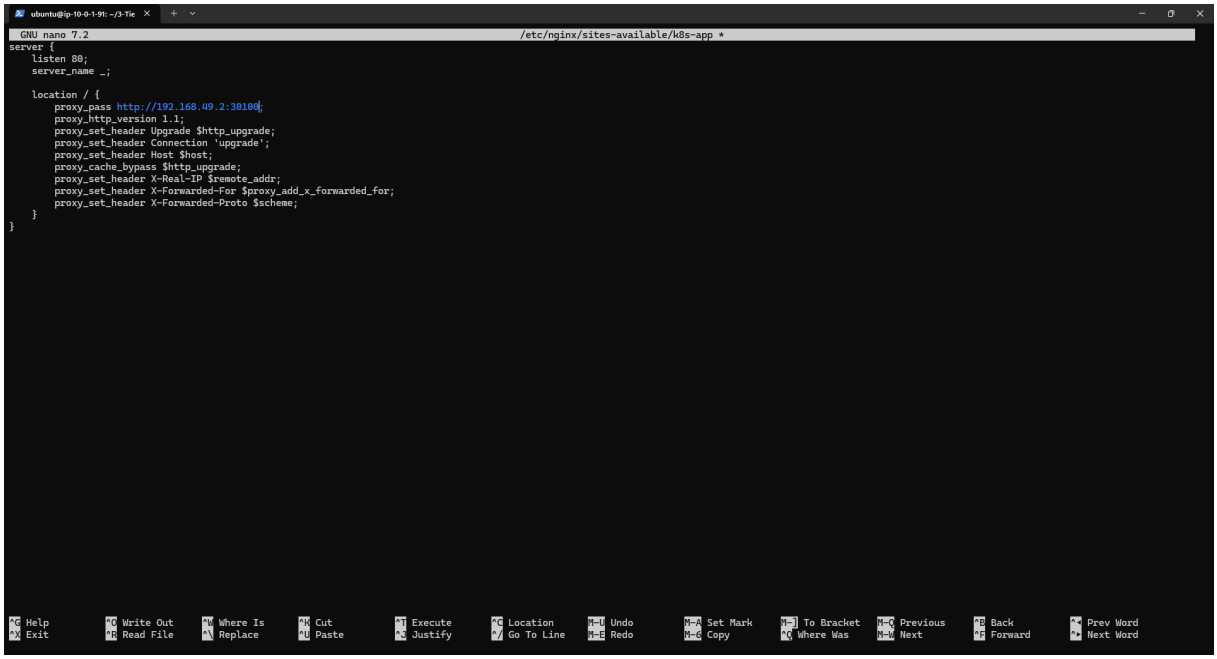


Figure 3 — Nginx site configuration with proxy_pass to the Minikube NodePort

Enabling the site

```
sudo ln -s /etc/nginx/sites-available/k8s-app /etc/nginx/sites-enabled/
sudo nginx -t                # validate configuration
sudo systemctl restart nginx
```

If the default site conflicts on port 80, remove it with `sudo rm /etc/nginx/sites-enabled/default` before restarting.

6. Verification

Browsing to the EC2 public IP over HTTP loads the **User Profile** page served by the web app pod, with the profile details (name, email, interests) read from MongoDB. This confirms that all three tiers communicate end to end.

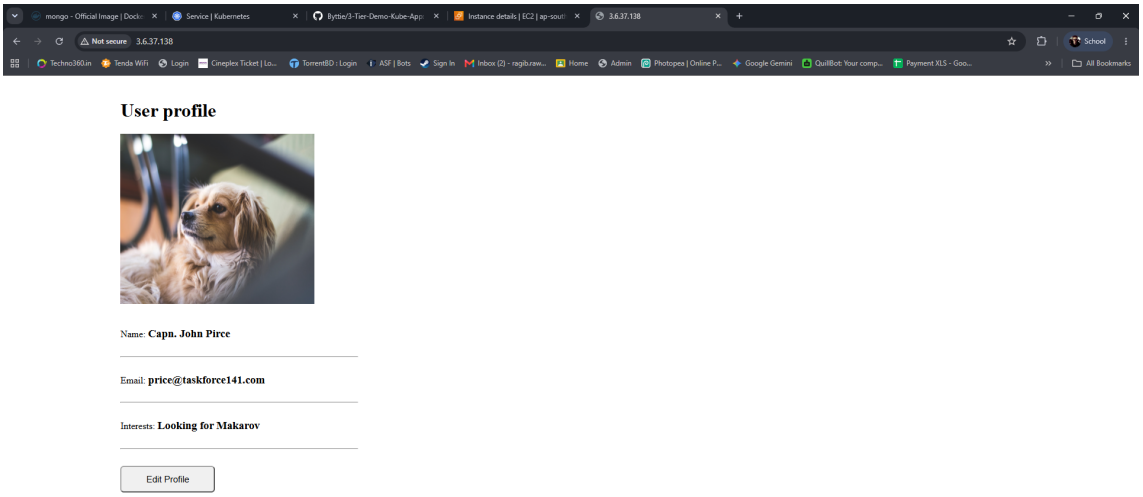


Figure 4 — The running application, reached through the EC2 public IP